

# Lab: 2D Convolution — RGB Image via AXI-Stream (Multi-Channel)

This notebook loads `cat.jpg` from the same directory, resizes it to fit the HLS constraints, and runs it through the `conv2d_stream` HLS IP with:

- **Cin = 3** (R, G, B channels)
- **Cout = 2** (edge detection + blur)
- **AXI-Stream** DMA for pixel data
- **AXI-Lite** for scalar registers and 3D kernel

## 1. Load Overlay

```
In [2]: import os
import time
import struct
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from pynq import Overlay, allocate
from scipy.signal import convolve2d

ol = Overlay("design_1.bit")
print("Overlay loaded")
print("IP blocks:", list(ol.ip_dict.keys()))

dma      = ol.axi_dma_0
conv2d    = ol.conv2d_stream_0
```

```
Overlay loaded
IP blocks: ['conv2d_stream_0', 'axi_timer_0', 'axi_dma_0', 'processing_system7_0']
```

## 2. Load & Preprocess `cat.jpg`

HLS constraints:

- **MAX\_WIDTH = 128** → both ROWS and COLS must be  $\leq 128$
- **MAX\_CIN = MAX\_COUT = 8**
- Kernel is fixed 3×3

The image is resized to fit within 128×128 while preserving aspect ratio, then padded with zeros to a square if needed. Pixel values are normalized to `[0.0, 1.0]` as `float32`.

```

In [3]: # — HLS hardware constants —————
MAX_WIDTH = 128
MAX_CIN    = 8
MAX_COUT   = 8
K          = 3

# — Channel config —————
CIN = 3    # RGB
COUT = 2   # edge detection + blur

# — Load image —————
IMAGE_PATH = os.path.join(os.getcwd(), "cat.jpg")
assert os.path.exists(IMAGE_PATH), f"Image not found: {IMAGE_PATH}"

pil_img = Image.open(IMAGE_PATH).convert("RGB")
print(f"Original size : {pil_img.size} (W x H)")

# — Resize to fit MAX_WIDTH=128 (preserve aspect ratio) —————
orig_w, orig_h = pil_img.size
scale = MAX_WIDTH / max(orig_w, orig_h)
new_w = int(orig_w * scale)
new_h = int(orig_h * scale)

# HLS requires even dimensions to be safe – round down to even
new_w = new_w if new_w % 2 == 0 else new_w - 1
new_h = new_h if new_h % 2 == 0 else new_h - 1

pil_resized = pil_img.resize((new_w, new_h), Image.LANCZOS)
print(f"Resized to      : {new_w} x {new_h} (W x H)")

COLS = new_w
ROWS = new_h

assert ROWS <= MAX_WIDTH, f"ROWS={ROWS} exceeds MAX_WIDTH={MAX_WIDTH}"
assert COLS <= MAX_WIDTH, f"COLS={COLS} exceeds MAX_WIDTH={MAX_WIDTH}"

# — Convert to float32 numpy array, normalize to [0, 1] —————
# PIL gives (H, W, 3) uint8 → we want (3, H, W) float32
img_np = np.array(pil_resized, dtype=np.float32) / 255.0 # (H, W, 3)
image_in = np.transpose(img_np, (2, 0, 1))                # (3, H, W)

print(f"image_in shape: {image_in.shape} (CIN x ROWS x COLS)")
print(f"Pixel range    : [{image_in.min():.3f}, {image_in.max():.3f}]")

# — Preview original image —————
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].imshow(pil_img)
axes[0].set_title(f"Original ({orig_w}x{orig_h})")
axes[1].imshow(pil_resized)
axes[1].set_title(f"Resized for HW ({COLS}x{ROWS})")
for ax in axes: ax.axis('off')
plt.tight_layout()
plt.show()

```

```

Original size : (525, 350) (W x H)
Resized to    : 128 x 84 (W x H)
image_in shape: (3, 84, 128) (CIN x ROWS x COLS)
Pixel range   : [0.000, 0.973]

```



### 3. Define Kernels

Kernel shape: (COUT, CIN, K\*K) — zero-padded to (MAX\_COUT, MAX\_CIN, K\*K) in HW.

| Output Channel | Filter                                                     |
|----------------|------------------------------------------------------------|
| oc=0           | Edge detection (Laplacian), applied equally across R, G, B |
| oc=1           | Box blur (3x3 average), applied equally across R, G, B     |

```
In [4]: kernels = np.zeros((COUT, CIN, K * K), dtype=np.float32)

# oc=0: Edge detection – sum over all RGB channels then detect edges
edge = np.array([-1,-1,-1, -1,8,-1, -1,-1,-1], dtype=np.float32)
for ic in range(CIN):
    kernels[0, ic] = edge / CIN    # normalize by Cin so output scale

# oc=1: Box blur – average over all RGB channels
blur = np.ones(K * K, dtype=np.float32) / (K * K)
for ic in range(CIN):
    kernels[1, ic] = blur / CIN    # normalize by Cin

print(f"kernels shape: {kernels.shape} (COUT x CIN x K*K)")
print(f"\nKernel[0][ic=0] (edge):\n{kernels[0,0].reshape(K,K)}")
print(f"\nKernel[1][ic=0] (blur):\n{kernels[1,0].reshape(K,K)}")
```

kernels shape: (2, 3, 9) (COUT x CIN x K\*K)

```
Kernel[0][ic=0] (edge):
[[-0.33333334 -0.33333334 -0.33333334]
 [-0.33333334  2.6666667  -0.33333334]
 [-0.33333334 -0.33333334 -0.33333334]]
```

```
Kernel[1][ic=0] (blur):
[[0.03703704 0.03703704 0.03703704]
 [0.03703704 0.03703704 0.03703704]
 [0.03703704 0.03703704 0.03703704]]
```

### 4. Software Golden Model

```
In [5]: def sw_conv2d(image_in, kernels, COUT, CIN, K):
        """Software reference matching HLS accumulation over all Cin channels
        _, ROWS, COLS = image_in.shape
        out = np.zeros((COUT, ROWS, COLS), dtype=np.float32)
        for oc in range(COUT):
            for ic in range(CIN):
                k2d = kernels[oc, ic].reshape(K, K)
                out[oc] += convolve2d(image_in[ic], k2d,
                                     mode='same', boundary='fill', fillval=0)
        return out

image_out_sw = sw_conv2d(image_in, kernels, COUT, CIN, K)
print(f"SW output shape: {image_out_sw.shape} (COUT x ROWS x COLS)")

SW output shape: (2, 84, 128) (COUT x ROWS x COLS)
```

## 5. Configure IP via AXI-Lite

| Register | Offset | Value                          |
|----------|--------|--------------------------------|
| ap_ctrl  | 0x00   | ap_start                       |
| rows     | 0x10   | ROWS                           |
| cols     | 0x18   | COLS                           |
| Cin      | 0x20   | 3                              |
| Cout     | 0x28   | 2                              |
| kernel   | 0x40   | kernel[MAX_COUT][MAX_CIN][K*K] |

Verify these offsets against your `xconv2d_stream_hw.h` from Vitis HLS export.

```

In [6]: CTRL_REG      = 0x0000
        ROWS_OFFSET   = 0x0010
        COLS_OFFSET   = 0x0018
        CIN_OFFSET    = 0x0020
        COUT_OFFSET   = 0x0028
        KERNEL_BASE   = 0x1000

        AP_START      = 0x01
        AP_DONE       = 0x02
        AP_IDLE       = 0x04

def float_to_uint(f):
    return struct.unpack('<I', struct.pack('<f', float(f)))[0]

# — Write scalar registers —————
conv2d.write(ROWS_OFFSET, ROWS)
conv2d.write(COLS_OFFSET, COLS)
conv2d.write(CIN_OFFSET, CIN)
conv2d.write(COUT_OFFSET, COUT)
print(f"Written: rows={ROWS}, cols={COLS}, Cin={CIN}, Cout={COUT}")

# — Write 3D kernel to HW memory map —————
# HW layout: kernel[MAX_COUT][MAX_CIN][K*K]
# Byte address = KERNEL_BASE + oc*(MAX_CIN*K*K*4) + ic*(K*K*4) + k*4
oc_stride = MAX_CIN * K * K * 4
ic_stride = K * K * 4

for oc in range(COUT):
    for ic in range(CIN):
        for k in range(K * K):
            addr = KERNEL_BASE + oc * oc_stride + ic * ic_stride + k
            conv2d.write(addr, float_to_uint(kernels[oc, ic, k]))

print(f"Kernel written: {COUT}x{CIN}x{K*K} = {COUT*CIN*K*K} floats")

```

Written: rows=84, cols=128, Cin=3, Cout=2

Kernel written: 2x3x9 = 54 floats

## 6. Prepare DMA Buffers

Input pixels are sent **channels-last** (interleaved per spatial pixel):

[ R(0,0), G(0,0), B(0,0), R(0,1), G(0,1), B(0,1), ... ]

This matches the HLS READ\_CHANNELS loop which reads `Cin` packets per valid `(pi, pj)`.

```
In [7]: # (CIN, ROWS, COLS) → (ROWS, COLS, CIN) → flatten [channels-last, row-major]
image_in_hwc = np.transpose(image_in, (1, 2, 0)).astype(np.float32)
in_flat      = image_in_hwc.flatten()

assert len(in_flat) == ROWS * COLS * CIN, "Input buffer size mismatch"

in_buf = allocate(shape=(ROWS * COLS * CIN,), dtype=np.float32)
out_buf = allocate(shape=(ROWS * COLS * COUT,), dtype=np.float32)

np.copyto(in_buf, in_flat)
out_buf[:] = 0

print(f"in_buf   : {len(in_buf)} words   ({ROWS}x{COLS}x{CIN})")
print(f"out_buf  : {len(out_buf)} words  ({ROWS}x{COLS}x{COUT})")

in_buf : 32256 words   (84x128x3)
out_buf : 21504 words  (84x128x2)
```

## 7. Execute DMA Transfer

```
In [19]: # — Safely (re)start DMA channels —————
for ch in [dma.sendchannel, dma.recvchannel]:
    try:
        ch.stop()
    except Exception:
        pass
    ch.start()

# — Assert IP is idle —————
status = conv2d.read(CTRL_REG)
assert status & AP_IDLE, f"IP not idle before start! CTRL={status:#04x}"

# — Start IP BEFORE DMA —————
conv2d.write(CTRL_REG, AP_START)

# — Transfer —————
t0 = time.perf_counter()

dma.recvchannel.transfer(out_buf) # arm recv first
dma.sendchannel.transfer(in_buf)

dma.sendchannel.wait()
dma.recvchannel.wait()

t_dma = time.perf_counter() - t0

# — Check IP completion —————
status = conv2d.read(CTRL_REG)
print(" IP ap_done" if (status & AP_DONE) else f" ap_done NOT set. CTRL={status:#04x}")
print(f"Processed {ROWS}x{COLS} RGB image (Cin={CIN}→Cout={COUT}) in {t_dma:.2f} ms")

IP ap_done
Processed 84x128 RGB image (Cin=3→Cout=2) in 127.14 ms
```

## 8. Unpack Hardware Output

```
In [9]: # out_buf is (ROWS*COLS*COUNT) interleaved channels-last
# Reshape → (ROWS, COLS, COUNT) → transpose → (COUNT, ROWS, COLS)
out_hwc = np.array(out_buf).reshape((ROWS, COLS, COUNT))
image_out_hw = np.transpose(out_hwc, (2, 0, 1)) # (COUNT, ROWS, COLS)

print(f"HW output shape: {image_out_hw.shape} (COUNT x ROWS x COLS)")
```

HW output shape: (2, 84, 128) (COUNT x ROWS x COLS)

## 9. Verification

```
In [20]: print("=" * 55)
print("Verification (inner window [1:-1, 1:-1] – border excluded)")
print("=" * 55)

all_pass = True
CHANNEL_NAMES = ["Edge Detection (oc=0)", "Box Blur (oc=1)"]
for oc in range(COUNT):
    hw_valid = image_out_hw[oc, 1:-1, 1:-1]
    sw_valid = image_out_sw[oc, 1:-1, 1:-1]
    max_diff = np.max(np.abs(hw_valid - sw_valid))
    result = "PASS " if max_diff < 1e-4 else "FAIL "
    print(f" {CHANNEL_NAMES[oc]:25s} max|HW-SW| = {max_diff:.2e} ")
    if max_diff >= 1e-4:
        all_pass = False

print()
print("Overall:", "PASS " if all_pass else "FAIL – check register of")
```

```
=====
Verification (inner window [1:-1, 1:-1] – border excluded)
=====
Edge Detection (oc=0)      max|HW-SW| = 7.15e-07 [PASS ]
Box Blur (oc=1)           max|HW-SW| = 2.38e-07 [PASS ]
```

Overall: PASS

## 10. Visualization

Row 0: Original RGB image + per-channel input preview

Row 1: Software output per output channel

Row 2: Hardware output per output channel

```

In [18]: CHANNEL_NAMES = ["Edge Detection", "Box Blur"]
INPUT_NAMES = ["Red Channel", "Green Channel", "Blue Channel"]

# — Figure 1: Input image – full RGB + per-channel true colour —
fig, axes = plt.subplots(1, CIN + 1, figsize=(5 * (CIN + 1), 4))

# Full RGB composite
axes[0].imshow(np.transpose(image_in, (1, 2, 0))) # (CIN,H,W) → (H,W,CIN)
axes[0].set_title("Input: cat.jpg (RGB)")

# Per channel: zero out the other two channels so colour is pure R /
for ic in range(CIN):
    rgb_single = np.zeros((ROWS, COLS, 3), dtype=np.float32)
    rgb_single[:, :, ic] = image_in[ic] # ic=0 → Red, ic=1 → Green, ic=2 → Blue
    axes[ic + 1].imshow(rgb_single)
    axes[ic + 1].set_title(INPUT_NAMES[ic])

for ax in axes: ax.axis('off')
plt.suptitle("Input Image", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# — Figure 2: SW vs HW per output channel (grayscale is fine here)
fig, axes = plt.subplots(COUT, 2, figsize=(10, 5 * COUT))
if COUT == 1:
    axes = axes[np.newaxis, :]

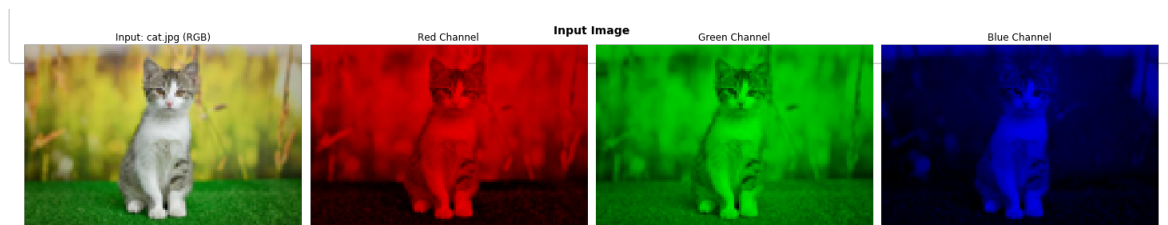
for oc in range(COUT):
    vmin = min(image_out_sw[oc].min(), image_out_hw[oc].min())
    vmax = max(image_out_sw[oc].max(), image_out_hw[oc].max())
    axes[oc, 0].imshow(image_out_sw[oc], cmap='gray', vmin=vmin, vmax=vmax)
    axes[oc, 0].set_title(f"SW: {CHANNEL_NAMES[oc]}")
    axes[oc, 1].imshow(image_out_hw[oc], cmap='gray', vmin=vmin, vmax=vmax)
    axes[oc, 1].set_title(f"HW: {CHANNEL_NAMES[oc]}")

for ax in axes.flat: ax.axis('off')
plt.suptitle("Software vs Hardware Output", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

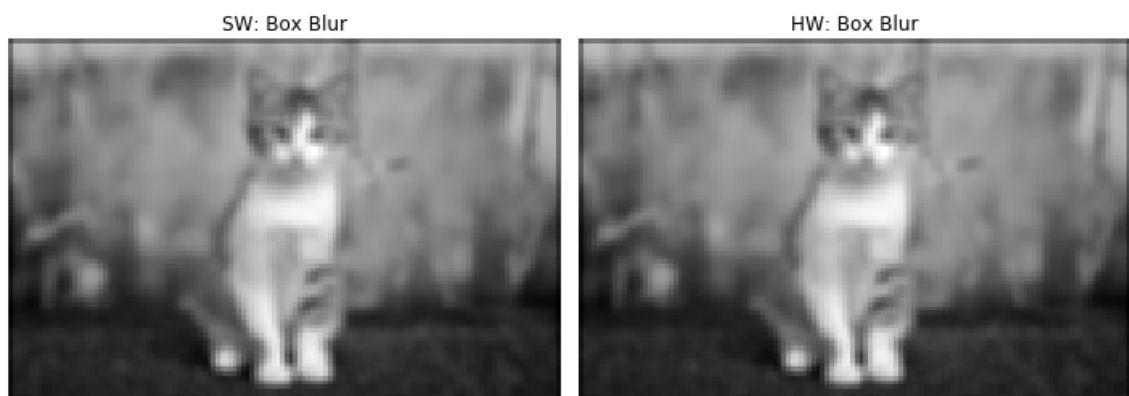
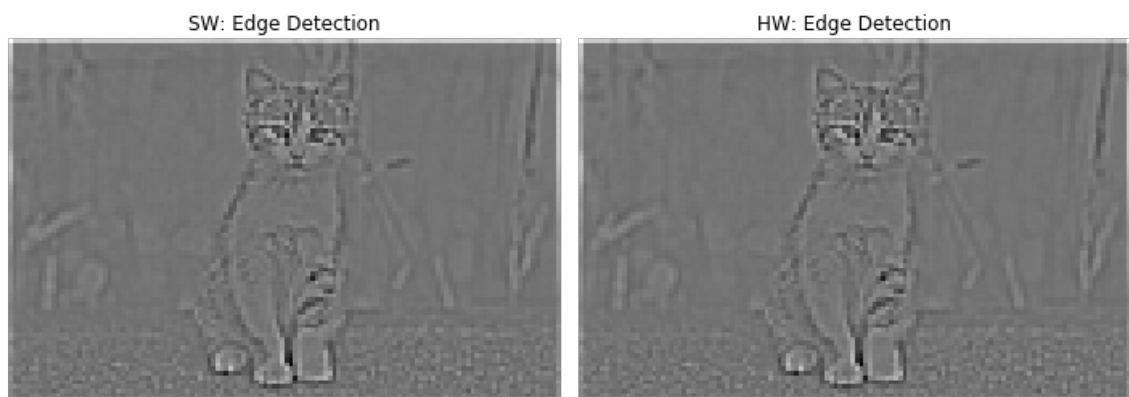
# — Figure 3: Final RGB composite from HW output channels —
def norm(arr):
    mn, mx = arr.min(), arr.max()
    return (arr - mn) / (mx - mn + 1e-8)

if COUT >= 3:
    # True RGB: one output channel maps directly to R, G, B
    rgb_hw = np.stack([norm(image_out_hw[0]),
                       norm(image_out_hw[1]),
                       norm(image_out_hw[2])], axis=-1)
else:
    # False-colour: edge→Red, blur→Blue, Green=0
    r = norm(image_out_hw[0])
    g = np.zeros((ROWS, COLS), dtype=np.float32)
    b = norm(image_out_hw[1]) if COUT > 1 else np.zeros_like(r)
    rgb_hw = np.stack([r, g, b], axis=-1) # (H, W, 3)

```



### Software vs Hardware Output



## 11. Save Output Images (Optional)

```
In [12]: def save_output(arr, filename):
          """Normalize float output to uint8 and save as PNG."""
          mn, mx = arr.min(), arr.max()
          if mx > mn:
              normalized = ((arr - mn) / (mx - mn) * 255).astype(np.uint8)
          else:
              normalized = np.zeros_like(arr, dtype=np.uint8)
          Image.fromarray(normalized).save(filename)
          print(f"Saved: {filename}")

          save_output(image_out_hw[0], "cat_hw_edge.png")
          save_output(image_out_hw[1], "cat_hw_blur.png")
          save_output(image_out_sw[0], "cat_sw_edge.png")
          save_output(image_out_sw[1], "cat_sw_blur.png")

          Saved: cat_hw_edge.png
          Saved: cat_hw_blur.png
          Saved: cat_sw_edge.png
          Saved: cat_sw_blur.png
```

## 12. Cleanup

```
In [ ]: in_buf.freebuffer()
          out_buf.freebuffer()
          print("Buffers freed.")
```